

centralised in the software-based controller, which further pushes rules to the simple packet forwarding devices, i.e., data plane. SDN uses a centralised model for the control plane.

Architecture

Figure 2 depicts the SDN architecture. We can see it has been divided into three layers -- the application layer, the control layer, and the infrastructure layer (the data layer). The application layer and control layer communicate with each other via northbound interface APIs, whereas the control layer and data layer do so via southbound interface APIs.

The southbound interface enables this via the usage of the OpenFlow protocol. The interface provides:

- Programmatic control of all forwarding operations
- Capabilities to advertise
- Statistics reporting
- Event notifications

Northbound interface APIs enable communication between SDN-enabled applications and the control layer. This interface provides an abstract view of the network, which projects the latter's requirements and behaviour. An open and standard northbound interface facilitates portability and interoperability between the controller and applications.

Controllers

With the advent of SDN, many controllers came to light to dictate the forwarding rules of routing procedures. SDN sits at the control layer and plays an integral part in dictating rules and policies for the data flowing between the application layer and the data layer. Network devices in the SDN network have no intelligence of their own. To balance that, we need a central intelligence to perform the integral network operations and that's where the SDN controller comes into the picture. It is the central brain of the network, which has an abstract view of the topologies around and, accordingly, makes decisions for every data packet. It translates the requirements of SDN applications straight from the application layer to the data layer.

There are different types of SDN controllers, and we can choose from them based on their features. Table 1 showcases some of the features of different controllers.

Open vSwitch

Open vSwitch (OVS) was introduced to overcome the networking challenges of virtual machines. The primary purpose of OVS is to decouple virtual networks from their underlying physical networks. OVS is a software

implementation of a multilayer virtual switch. It provides a general switching stack for virtualised hardware environments, and supports multiple protocols and networking standards while enabling network automation via its programmatic extensions. Some of its features are:

- Visibility into inter-VM communication via NetFlow, sFlow(R), IPFIX, SPAN, RSPAN, and GRE-tunnelled mirrors
- Multi-cast snooping
- Fine-grained QoS control
- OpenFlow protocol support (including many extensions for virtualization)
- IPv6 support
- Multiple tunnelling protocols (GRE, VXLAN, STT, and Geneve, with IP security support)
- Multi-table forwarding pipeline with flow-caching engine

Mininet

Mininet is an emulator that can be used to create a network of hosts with links and switches. With Mininet, one can create virtual networks while experimenting with software defined networking systems. It enables rapid prototyping of SDN using OpenFlow. Some of the features of Mininet are:

- Command-line interface which instantiates networks

Controller name	NOX	POX	RYU	Floodlight	OpenDayLight
Language	C++	Python	Python	Java	Java
Performance	High	Low	Low	High	High
Distributed	No	No	Yes	Yes	Yes
OpenFlow support version	1.0	1.0	1.0-1.4	1.0	1.3
Cloud support	No	No	Yes	Yes	Yes
Learning	Medium	Easy	Medium	Hard	Hard

Table 1: Features of controllers